

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

75

Code of Conduct:

I Santhosh K / (192312450) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Santhosh K
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	4	4	3	2	16
Q2	3	4	2	3	1.5	13.5
Q3	4	3	2	3	2	14
Q4	3	5	4	4	1.5	17.5
Q5	2	5	3	2	2	14

(75)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	4 Clearly understands the problem and accurately identifies all requirements	3 Good understanding with minor gaps	2 Basic understanding; some requirements unclear	1 Poor understanding or misinterpretation of the problem
Application of Concepts	30	6 Accurately applies appropriate concepts to solve complex problems	5 Applies relevant concepts correctly with minor errors	4 Applies basic concepts with limited accuracy	1-2 Fails to apply appropriate concepts
Problem-Solving Approach	20	4 Logical, well-structured, and efficient approach	3 Mostly logical approach with minor gaps	2 Basic approach with some inconsistencies	1 Illogical or unclear approach
Accuracy of Solution	20	4 Completely correct solution with proper justification	3 Mostly correct with minor errors	2 Partially correct solution	1 Incorrect or incomplete solution
Clarity	10	2 Clear, well-organized, and properly explained solution	1.5 Understandable with minor clarity issues	1 Limited clarity and explanation	0 Poorly presented and difficult to understand

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Smart City Surveillance System – CPU, Memory, I/O and Bus Structure

Theory and Problem Analysis

A computer system consists mainly of the CPU, memory, I/O devices and buses. The CPU executes instructions, memory stores programs and data, and I/O devices communicate with external devices. The system bus carries data, addresses and control signals between these units.

In a smart city surveillance system, hundreds of cameras may continuously transmit video. During peak hours, the amount of data increases and CPU, memory and I/O devices compete for communication resources. This can create **bus congestion and memory delays**, causing slower threat detection.

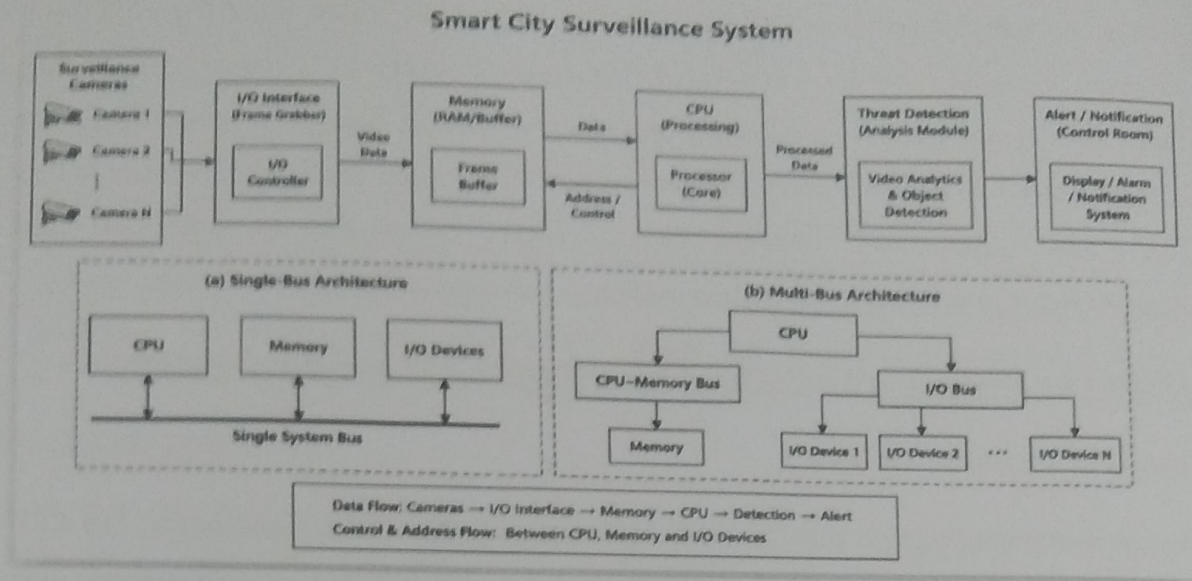
How It Works in the Real World

Consider a traffic-junction CCTV camera. The camera continuously captures vehicles and people and sends video frames through a high-speed I/O interface. The frames are temporarily stored in RAM. The CPU or another processing unit reads these frames and runs algorithms for **vehicle detection, number-plate recognition or suspicious-activity detection**.

The practical data flow is:

Camera → I/O Interface → RAM → CPU Processing → Detection → Alert

If many cameras send data through a **single bus**, they compete for the same communication path. With a **multi-bus architecture**, I/O transfers and CPU-memory communication can use separate paths, reducing waiting time. DMA can further transfer camera data directly to memory without requiring the CPU to manage every byte.



Advantages

- Multi-bus architecture provides higher bandwidth.
- DMA reduces CPU involvement in data transfer.
- Cache memory reduces memory-access time.
- Pipelining improves instruction throughput.

Disadvantages and Challenges

- Multi-bus architecture increases hardware cost.
- High-resolution video requires large memory bandwidth.
- Many simultaneous cameras can create I/O congestion.
- Real-time processing requires high CPU performance.

Performance

$$\text{CPU Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate}$$

For 1×10^9 instructions, CPI = 4 and clock rate = 2 GHz:

$$\text{CPU Time} = 2 \text{ seconds}$$

Reducing CPI to 2 reduces CPU time to **1 second**.

Real-Time Applications

Smart city CCTV, traffic monitoring, airport security, face recognition and industrial surveillance.

Solution and Conclusion

Using multi-bus architecture, DMA, cache, pipelining and multicore processing allows camera data to move and process faster. Therefore, the system can detect threats with lower delay.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Real-Time Stock Trading – Instruction Cycle and CPI

Theory and Problem Analysis

A CPU executes instructions using the **fetch-decode-execute** cycle. Modern processors generally divide this into **Fetch → Decode → Execute → Memory Access → Write Back**. CPI represents the average number of clock cycles required for an instruction.

In a stock trading system, millions of instructions may be executed when market activity is high. Frequent memory access, cache misses and branch instructions increase CPI and delay transactions.

How It Works in the Real World

Suppose a user places an order to buy a stock when its price falls below ₹500. The trading server continuously receives updated prices.

The CPU performs operations such as:

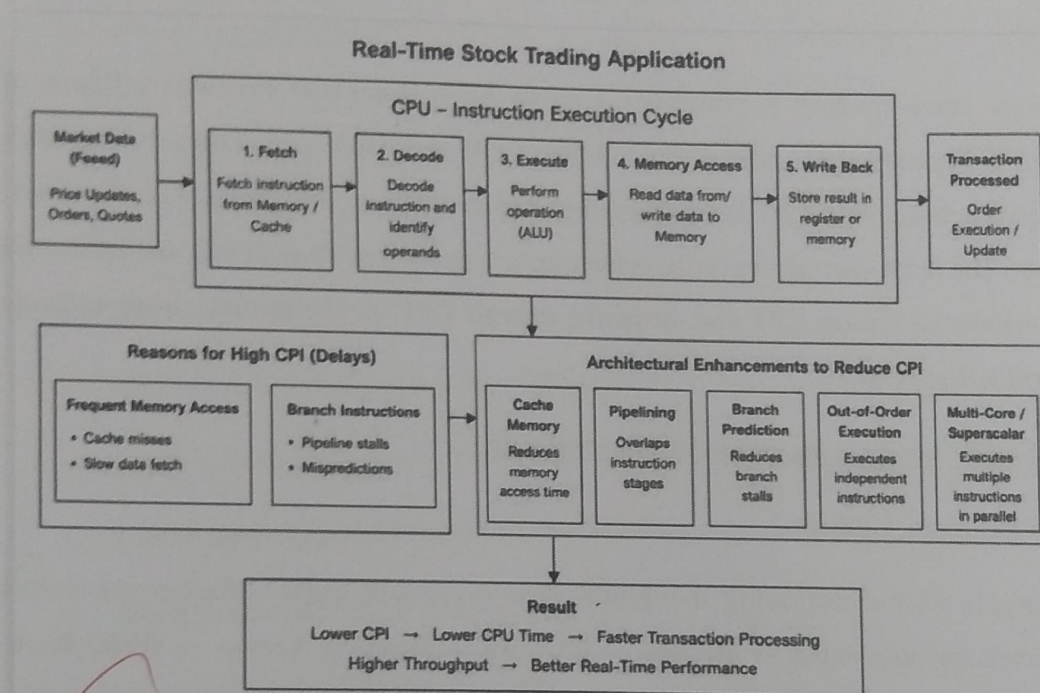
Receive Price → Fetch Instruction → Compare Price → Check Condition → Execute Buy Order → Update Account

For example:

If Price $\leq$ ₹500 $\rightarrow$ Buy

The CPU fetches the comparison instruction, decodes it and executes it. Since this condition involves a branch, the CPU must decide which instruction to execute next.

When millions of users place orders simultaneously, repeated memory accesses and branch instructions can create pipeline stalls. **Cache memory** keeps frequently required information closer to the CPU, while **branch prediction** predicts likely program paths and reduces waiting.



Performance

For 500 million instructions, CPI = 4 and clock rate = 2.5 GHz:

CPU Time = 0.8 seconds

If architectural improvements reduce CPI to 2:

CPU Time = 0.4 seconds

Advantages

- Cache provides faster data access.

- Pipelining improves instruction throughput.
- Branch prediction reduces stalls.
- Superscalar processing can execute multiple instructions simultaneously.

Disadvantages and Challenges

- Wrong branch predictions waste CPU cycles.
- Larger caches increase cost.
- Millions of simultaneous transactions create heavy CPU load.
- Trading systems require extremely low latency.

Real-Time Applications

Stock exchanges, UPI transactions, banking systems, cryptocurrency trading and e-commerce payments.

Solution and Conclusion

Using cache, pipelining, branch prediction, out-of-order execution and superscalar processing reduces CPI. As a result, buy/sell orders can be processed more quickly during high market activity.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

8085 Temperature Control System – Addressing Modes

Theory and Problem Analysis

An **addressing mode** specifies how the processor obtains the data required by an instruction. The 8085 supports immediate, register, direct, register-indirect and implicit addressing.

In a temperature-control system, incorrect addressing can cause the CPU to obtain data from the wrong memory location, resulting in incorrect temperature decisions.

Examples include:

MVI A,25H – Immediate addressing

MOV A,B – Register addressing

LDA 2500H – Direct addressing

MOV A,M – Register-indirect addressing

CMA – Implicit addressing

How It Works in the Real World

Consider an industrial machine that must operate below 50°C. A temperature sensor measures the machine temperature and sends the reading to the microprocessor through an input interface.

The operation is:

Temperature Sensor → Input Port → 8085 → Compare Temperature → Actuator

Suppose the measured temperature is 55°C. The 8085 reads the sensor using an IN instruction and compares the value with the 50°C threshold using CMP.

If:

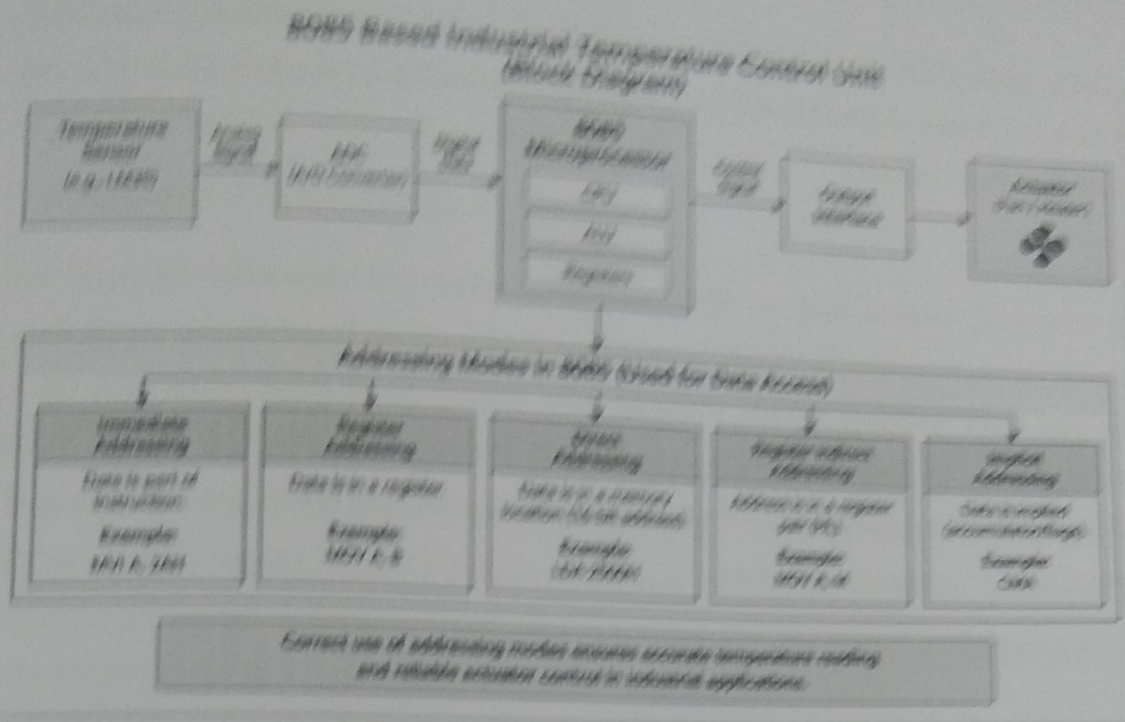
Temperature > 50°C → Turn ON cooling fan

Otherwise:

Temperature ≤ 50°C → Continue normal operation

The CPU can use OUT to send a control signal to the fan.

If the sensor value is stored at address 2500H but the program incorrectly accesses 2600H, the processor reads the wrong value. The fan may remain OFF even when the machine is overheating.



Advantages

- Register addressing provides fast access.
- Immediate addressing is useful for threshold values.
- Direct addressing is suitable for fixed memory locations.
- Register-indirect addressing supports multiple sensor values.

Disadvantages and Challenges

- Incorrect addresses cause incorrect readings.
- Wrong register initialization affects data access.
- Incorrect I/O port selection can control the wrong device.
- Sensor data must be read at the correct time.

Real-Time Applications

Industrial temperature controllers, automatic fans, boilers, motor protection and process-control systems.

Solution and Conclusion

The programmer must use correct addressing modes, initialize registers correctly and verify I/O addresses. Proper addressing ensures that the processor reads the actual sensor value and controls the correct actuator.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

8086 – Segmented Memory

Theory and Problem Analysis

The 8086 has a **20-bit address bus** and can access **1 MB of memory**. It uses segmentation because its main registers are 16-bit.

The major segments are:

- CS: Code Segment
- DS: Data Segment
- SS: Stack Segment
- ES: Extra Segment

Physical address is calculated as:

$$\text{Physical Address} = \text{Segment} \times 10\text{H} + \text{Offset}$$

For example:

$$\text{CS} = 2000\text{H} \text{ and } \text{IP} = 1500\text{H}$$

$$\text{Physical Address} = 20000\text{H} + 1500\text{H} = 21500\text{H}$$

How It Works in the Real World

Consider an industrial monitoring program running on an 8086. The program instructions may be stored in the **Code Segment**, sensor readings in the **Data Segment**, and temporary values or return addresses in the **Stack Segment**.

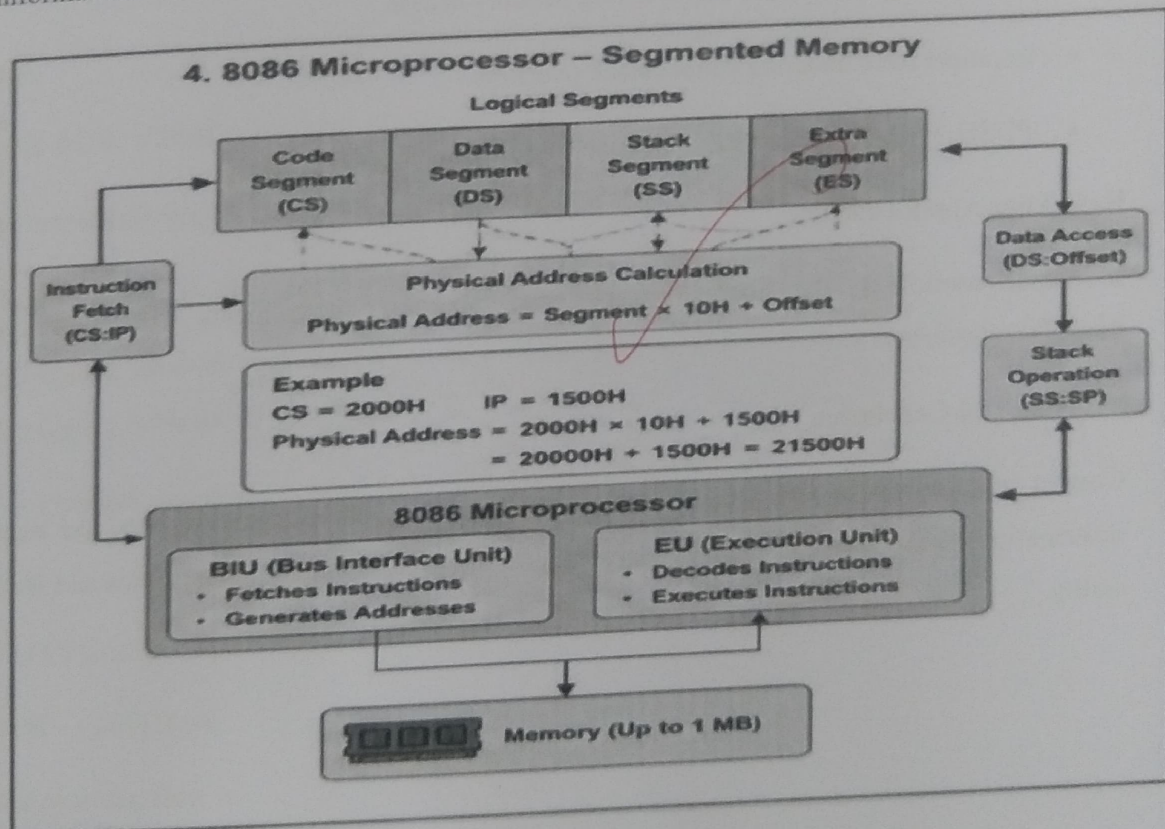
The operation can be represented as:

CS:IP → Fetch Program Instruction

DS:Offset → Access Sensor/Data Value

SS:SP → Store Temporary/Return Information

Suppose a program calls a temperature-monitoring function. The return address is placed on the stack. After completing the function, the processor retrieves this address and returns to the main program. If SS or SP is incorrect, the return address may be stored in the wrong location. The program may then return to an incorrect instruction or crash. Similarly, if DS points to the wrong segment, the program may read another program's data instead of the required sensor information.



Instruction Execution

The 8086 has a **Bus Interface Unit (BIU)** and **Execution Unit (EU)**. The BIU fetches upcoming instructions while the EU executes the current instruction. A **6-byte instruction queue** allows fetching and execution to overlap.

Advantages

- Supports up to 1 MB memory.
- Separates code, data and stack.
- Provides organized memory management.
- Instruction prefetching improves performance.

Disadvantages and Challenges

- Segment management is complex.
- Incorrect segment values cause wrong memory access.
- Improper stack handling causes stack overflow.
- PUSH, POP, CALL and RET must be carefully managed.

Real-Time Applications

Industrial controllers, data-acquisition systems, laboratory instruments and embedded monitoring systems.

Solution and Conclusion

Correct initialization of **CS, DS, SS, ES and SP** ensures that instructions, data and stack information are accessed correctly. Proper segmentation prevents memory corruption and stack errors.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Hexadecimal-Binary Conversion and Benchmarking

Theory and Problem Analysis

Computers internally process information using **binary**, but binary numbers are difficult for humans to read when they become long. Hexadecimal provides a shorter representation because each hexadecimal digit corresponds to **four binary bits**.

For example:

$$3AH = 0011\ 1010_2$$

and

$$1011\ 0110_2 = B6H$$

Incorrect conversion changes the actual value processed by the CPU.

How It Works in the Real World

Consider a network packet received by a computer. Packet information may be displayed by debugging software in hexadecimal.

For example, suppose a control field contains:

3AH

The CPU actually processes its binary representation:

$$3AH \rightarrow 00111010_2$$

The practical flow is:

Network Device → Packet → Hexadecimal Representation → Binary Processing → CPU → Required Action

Suppose the engineer incorrectly interprets 3AH as 00110110 instead of 00111010. The CPU receives a different value.

Correct:

$$00111010_2 = 58$$

Incorrect:

$$00110110_2 = 54$$

This can change a **packet command, device address, data length or control field**, resulting in incorrect processing.

CPU Performance and Benchmarking

CPU performance is calculated using:

$$\text{CPU Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate}$$

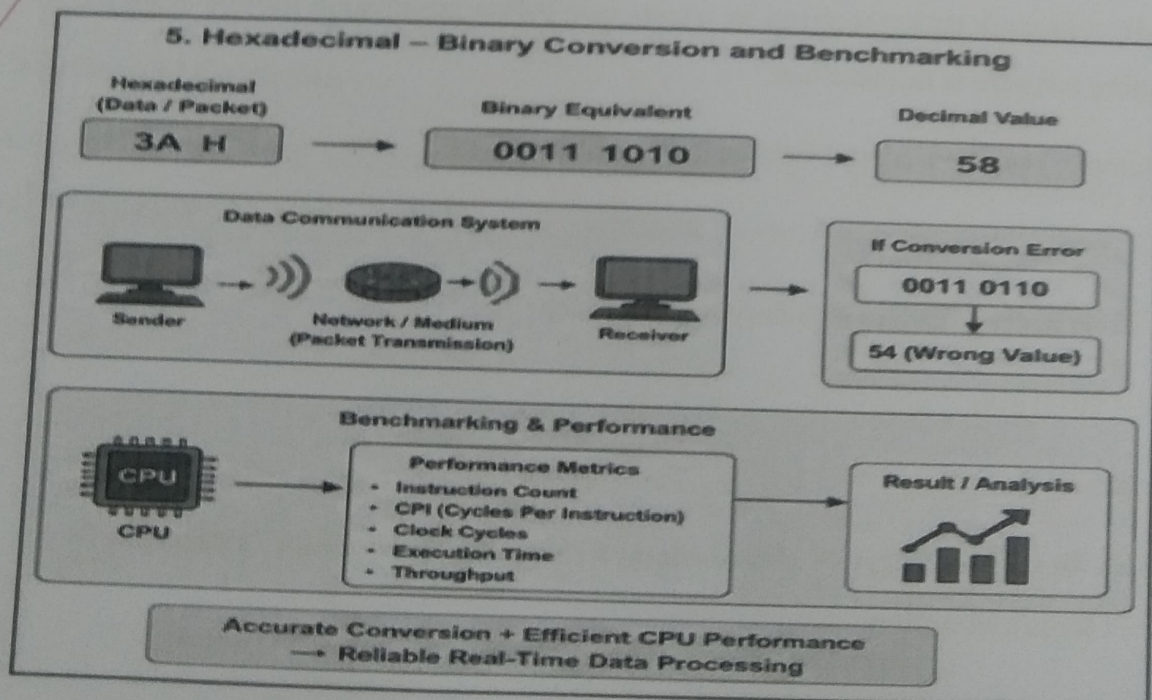
For 20 million instructions, CPI = 3 and clock rate = 2 GHz:

$$\text{CPU Time} = 0.03 \text{ seconds}$$

If CPI is reduced to 1.5:

$$\text{CPU Time} = 0.015 \text{ seconds}$$

Therefore:



Advantages

- Hexadecimal is shorter and easier to debug than binary.
- Conversion between hexadecimal and binary is simple.
- Benchmarking identifies performance bottlenecks.
- Error-detection techniques improve reliability.

Disadvantages and Challenges

- Incorrect conversion changes data values.
- A single-bit error may change packet information.
- Large packet volumes increase CPU and memory load.
- Real-time communication requires very low processing delay.

Real-Time Applications

Network routers, Ethernet, Wi-Fi, IoT devices, automotive networks, embedded communication and network debugging.

Solution and Conclusion

Automatic hexadecimal-binary conversion reduces human errors, while **CRC, checksum and packet validation** detect corrupted information. Efficient cache, memory and CPU architecture improve packet-processing speed, and benchmarking helps identify performance bottlenecks. Therefore, these techniques improve the reliability of real-world communication systems.